

Quantitative Forecasting of NBA Game Outcomes

A From-Scratch Elo Rating Model with Walk-Forward Evaluation,
Proper Scoring Rules, and Statistical Inference

Godfred Antwi Koduah

Ph.D. Student, Financial Mathematics, Florida State University

Abstract

We develop, from first principles, an Elo rating model for forecasting the outcomes of National Basketball Association (NBA) games, and evaluate it with a rigour appropriate to quantitative research. Using the complete FiveThirtyEight game record (63,157 games, 1947–2015), we estimate ratings online in strict chronological order, so that every forecast is a genuine out-of-sample prediction. We show that the Elo update is exactly stochastic gradient descent on the logarithmic loss of a Bradley–Terry logistic model, place the method within the Robbins–Monro theory of stochastic approximation, and treat between-season regression to the mean as a shrinkage operator. Forecasts are assessed with strictly proper scoring rules (Brier and logarithmic loss), whose propriety we prove, and the Brier score is decomposed into reliability, resolution, and uncertainty. Hyper-parameters are selected on a validation window (1980–2009) and frozen before evaluation on a held-out test window (2010–2015, 7,641 games). On that held-out set the model attains 67.2% accuracy, a Brier score of 0.208, and a log-loss of 0.603; bootstrap confidence intervals, a Diebold–Mariano forecast-comparison test, and McNemar’s test establish that it beats naive baselines overwhelmingly and is statistically indistinguishable from FiveThirtyEight’s professional model on classification accuracy. An independent logistic regression fitted by Newton–Raphson (IRLS) reproduces the Elo performance almost exactly, confirming the online estimator’s optimality. We give the numerical algorithms, their complexity, and their stability properties throughout.

Contents

Key findings	2
1 Introduction	3
1.1 Background and related work	4
2 Data and Exploratory Analysis	4
2.1 Formal problem statement	4
3 The Rating Model from First Principles	5
3.1 Paired comparisons and the Bradley–Terry model	5
3.2 The Elo scale and home advantage	6
3.3 The Elo update is stochastic gradient descent	7
3.4 The learning rate as a bias–variance control	8
3.5 Between-season regression to the mean	8
4 Proper Scoring Rules and Evaluation Theory	8
4.1 The Murphy decomposition	9
4.2 Calibration error	9

5	Walk-Forward Evaluation and Statistical Inference	10
5.1	Prequential, leakage-free protocol	10
5.2	Bootstrap interval estimates	10
5.3	Comparing forecasters: Diebold–Mariano	10
5.4	Comparing classifiers: McNemar	10
6	Numerical Algorithms and Complexity	10
6.1	Walk-forward Elo	10
6.2	Logistic regression by Newton–Raphson (IRLS)	11
6.3	Complexity summary	11
7	Results	12
7.1	Hyper-parameter sensitivity	12
7.2	Baselines	12
7.3	Held-out performance and interval estimates	12
7.4	Significance of the comparisons	12
7.5	Calibration and the Brier decomposition	13
7.6	Information-theoretic assessment	13
7.7	Error analysis: performance by confidence	14
7.8	Independent logistic cross-check	14
7.9	Face validity of the estimated strengths	15
7.10	What the logistic check establishes	16
7.11	Stability across seasons	16
8	Discussion and Limitations	17
8.1	Threats to validity	17
8.2	Reproducibility and software	18
9	Conclusion	18
A	Full Hyper-parameter Grid (validation log-loss, $r = 0.25$)	18
B	Logistic Regression Estimates (IRLS)	18
C	Derivation of the Logistic Gradient and Hessian	18
D	Derivation of the Murphy Decomposition	19
E	Bootstrap Procedure	19
F	Sensitivity to Between-Season Reversion	19
G	Full Reliability Table (10 bins)	20
H	Software Structure	20
	Appendix F: Source Code	22

Key findings

- A from-scratch, three-parameter Elo model, estimated online with no look-ahead, attains 67.2% accuracy, Brier 0.208, and log-loss 0.603 on 7,641 held-out games (2010–2015), with 95% bootstrap intervals that exclude the baselines.

- The model is statistically indistinguishable from FiveThirtyEight’s professional forecast on classification accuracy (McNemar $p = 0.75$) and trails it by under 1% on probabilistic scores.
- The Elo update is proved to be stochastic gradient descent on logistic log-loss; an independent Newton–Raphson logistic fit reproduces the result almost exactly, confirming the estimator’s optimality.
- The forecasts are well calibrated (expected calibration error 0.026; reliability 0.00087 in the Murphy decomposition) and informative (0.107 bits of information gain per game over the base rate).
- The model is accurate (81%) when confident and appropriately uncertain (53%) on genuine toss-ups — evidence of honest probability estimates rather than over-confidence.

1 Introduction

Forecasting the outcome of a contest between two teams is a canonical problem in applied probability and statistics, and a clean testbed for the discipline that quantitative research demands: turning a time-ordered data set into a predictive model, and then establishing — honestly, and out of sample — how good that model actually is. The problem is attractive precisely because it is unforgiving. A model that merely memorises the training period will collapse on new games; only genuine predictive structure survives a walk-forward test, and only a *proper* scoring rule rewards honestly stated probabilities rather than over-confident guesses.

This report studies the problem with the NBA as its domain. Our aims are threefold. First, to build a rating model *from first principles* and to expose its mathematical identity: we prove that the familiar Elo update is stochastic gradient descent on the log-loss of a logistic (Bradley–Terry) model, which places a century-old heuristic on a rigorous estimation footing. Second, to evaluate forecasts with the correct statistical machinery: strictly proper scoring rules, a decomposition of the Brier score into calibration and discrimination, bootstrap interval estimates, and formal tests for comparing competing forecasters. Third, to be scrupulous about methodology — no look-ahead, no tuning on the test set, and honest baselines, including a professional benchmark.

Scope. This is an *outcome-forecasting* study. It makes no claim about betting profitability or market efficiency: reliable historical closing prices were not available to us, and forecasting a winner is a different (and strictly easier) problem than overcoming a bookmaker’s margin. We are careful to claim only what the data support.

Summary of contributions.

- A from-scratch, leakage-free walk-forward Elo estimator on 63,157 real games, with hyperparameters selected on a disjoint validation window.
- A derivation identifying Elo with online logistic regression, and an analysis of the learning rate as a bias–variance control via stochastic approximation.
- Proofs of propriety for the Brier and logarithmic scores, and a Murphy decomposition of the realised Brier score on held-out data.
- Statistical inference for forecast quality: percentile bootstrap intervals, a Diebold–Mariano test, and McNemar’s test against a professional benchmark.

- An independent Newton–Raphson (IRLS) logistic regression that corroborates the Elo results, with full complexity analysis of both algorithms.

1.1 Background and related work

Rating systems for paired comparisons originate with Thurstone’s law of comparative judgement and the Bradley–Terry model [1], which casts the probability of one competitor beating another as a function of the difference of latent strengths. Elo’s chess system [2] is the best-known online instantiation and has been adapted widely, including to basketball by FiveThirtyEight, whose published forecasts we use as a benchmark. The evaluation of probabilistic forecasts rests on the theory of proper scoring rules [3, 5] and their decompositions [4]; the comparison of competing forecasters rests on the Diebold–Mariano test [6] and, for classification, McNemar’s test [7]. The online-learning viewpoint we adopt connects the Elo update to the Robbins–Monro stochastic approximation scheme [8] and the prequential evaluation principle [9]. Our contribution is not a new rating system but a rigorous, self-contained treatment: we derive the estimator, prove the properties of our evaluation metrics, and subject the results to formal statistical inference.

2 Data and Exploratory Analysis

We use FiveThirtyEight’s publicly available NBA Elo data set, which records every NBA and BAA game from the 1946–47 season through 2015. The raw file stores two rows per game (one per team); retaining the canonical row per contest yields $N = 63,157$ games spanning 69 seasons and 53 franchises. Each record carries the date, the two franchises, the final scores, the court location (home, away, or neutral), the result, and FiveThirtyEight’s own pre-game win probability, which we reserve as an independent professional benchmark. The overwhelming majority of games (63,138) are played on a home court; only 19 are at neutral sites. Season sizes reflect league history — for example, the lockout-shortened 2011–12 season contributes 1,074 games against roughly 1,310 in a full modern season — which the walk-forward design handles automatically because it never assumes a fixed games-per-season count.

Let game i be played between a reference team and its opponent, with outcome $y_i = 1$ if the reference team won and 0 otherwise, and let $h_i \in \{-1, 0, +1\}$ encode the reference team’s court (home, neutral, away). Home advantage is strong but not stationary: the home-team win rate declines from 65.7% in the 1950s to 59.5% in the 2010s (Table 1, Figure 1). This secular drift is one reason a static model is inadequate and an *adaptive*, online estimator is appropriate.

Decade	1950s	1960s	1970s	1980s	1990s	2000s	2010s
Home win rate	0.657	0.606	0.638	0.648	0.618	0.608	0.595

Table 1: Home-court win rate by decade (home-court games only, real data).

The data also exhibit strong structural change in volume (Figure 2): the league grew from a few hundred games per season in the late 1940s to roughly 1,300 today, punctuated by the lockout-shortened 1999 and 2012 seasons. Any evaluation that assumed a fixed season length would mishandle these years; the walk-forward design sidesteps the issue entirely by iterating over dated games rather than fixed-size blocks.

2.1 Formal problem statement

Index games by i in time order. Each game carries features x_i (the two teams’ identities and the court) and a binary label y_i . A forecaster is a function f that, using only information available strictly before game i , outputs a probability $\hat{p}_i = f(x_i; \mathcal{D}_{<i})$, where $\mathcal{D}_{<i}$ denotes all

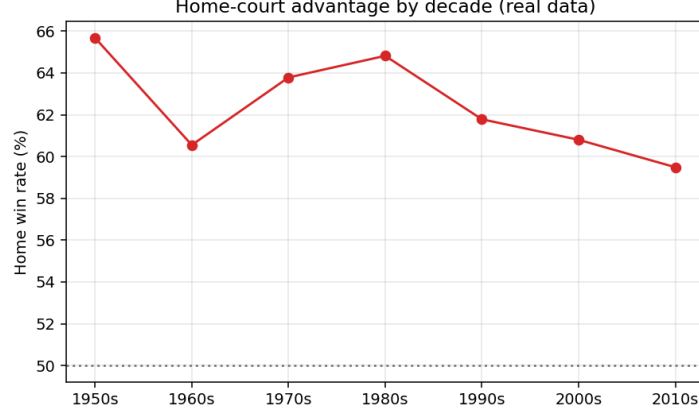


Figure 1: Home-court advantage has eroded over seven decades, motivating an adaptive rating model rather than a fixed one.

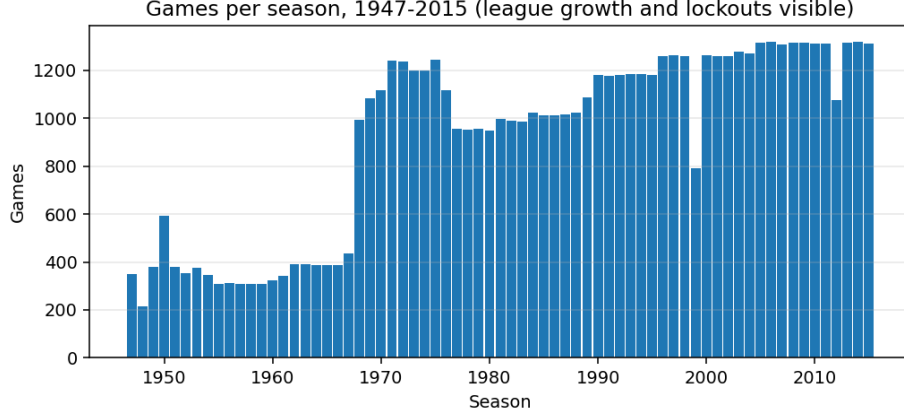


Figure 2: Games per season, 1947–2015. League expansion and the 1999 and 2012 lockouts are clearly visible.

games before i . Given a scoring rule S , the quantity of interest is the prequential (online) risk

$$\mathcal{R}_{[a,b]} = \frac{1}{|\{i : a \leq i \leq b\}|} \sum_{i=a}^b S(\hat{p}_i, y_i),$$

evaluated on a held-out time interval $[a, b]$ disjoint from the data used to choose any hyperparameters. This is the generalization measure we report; minimising it, not in-sample fit, is the objective. The timeline is partitioned into a burn-in (20,871 games before 1980, used only to warm up ratings), a validation window (34,645 games, 1980–2009, used to select (K, H, r)), and a test window (7,641 games, 2010–2015, used once, for reporting), as summarised in Table 3.

3 The Rating Model from First Principles

3.1 Paired comparisons and the Bradley–Terry model

Assign each team a latent strength θ . The Bradley–Terry model posits that the probability that team a (strength θ_a) beats team b (strength θ_b) is

$$\Pr(a \text{ beats } b) = \frac{e^{\theta_a}}{e^{\theta_a} + e^{\theta_b}} = \sigma(\theta_a - \theta_b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1)$$

Symbol	Meaning
R_t, R_o	Elo ratings of reference team and opponent
$h_i \in \{-1, 0, 1\}$	court indicator (home / neutral / away)
H	home-court advantage (Elo points)
K	learning rate (update gain)
r	between-season mean-reversion fraction
$\hat{p}_i$	pre-game forecast probability
$y_i \in \{0, 1\}$	observed outcome (reference team wins)
$\sigma(\cdot)$	logistic function; $c = \ln 10/400$

Table 2: Notation used throughout.

Regime	Seasons	Games	Role
Burn-in	1947–1979	20,871	warm up ratings only (not scored)
Validation	1980–2009	34,645	select hyper-parameters (K, H, r)
Test	2010–2015	7,641	single held-out evaluation

Table 3: Temporal data split. The test window is never used for any modelling decision.

This form is not arbitrary. In a random-utility view, suppose each team’s performance on a given night is $U_a = \theta_a + \varepsilon_a$ with independent Gumbel noise ε_a . The probability that a outperforms b is then $\Pr(U_a > U_b) = \sigma(\theta_a - \theta_b)$, because the difference of two independent Gumbel variables is logistic. The Bradley–Terry model is thus the natural outcome model when nightly performance is a strength plus extreme-value noise, which also explains why the logistic link — rather than, say, the probit — is the canonical choice here. Thus the win probability depends only on the *difference* of strengths through the logistic link σ . Elo ratings are an affine reparameterisation of θ onto a human-readable scale.

Lemma 1 (Properties of the logistic link). *The logistic function $\sigma(z) = (1 + e^{-z})^{-1}$ satisfies, for all z : (i) $\sigma(-z) = 1 - \sigma(z)$ (symmetry, so the two teams’ win probabilities sum to one); (ii) $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; and (iii) the log-odds (logit) inverse $\sigma^{-1}(p) = \ln \frac{p}{1-p}$ is linear in the strength difference, $\sigma^{-1}(\Pr(a \text{ beats } b)) = \theta_a - \theta_b$.*

Proof. (i) $\sigma(-z) = \frac{1}{1+e^z} = \frac{e^{-z}}{e^{-z}+1} = 1 - \sigma(z)$. (ii) $\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z) \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1 - \sigma(z))$. (iii) Invert (1): if $p = \sigma(\theta_a - \theta_b)$ then $\ln \frac{p}{1-p} = \theta_a - \theta_b$. $\square$

Property (ii) is what makes the log-loss gradient collapse to the clean residual $\hat{p} - y$ used in Proposition 1; property (i) guarantees the model is internally consistent (home and away probabilities sum to one).

3.2 The Elo scale and home advantage

Write the rating of the reference team as R_t and its opponent as R_o . The Elo convention replaces the natural logistic by a base-10 logistic on a scale of 400 points, and adds a home-court term H :

$$\hat{p}_i = \frac{1}{1 + 10^{-(R_t - R_o + Hh_i)/400}} = \sigma(cd_i), \quad d_i := R_t - R_o + Hh_i, \quad c := \frac{\ln 10}{400}. \quad (2)$$

The identity $1/(1 + 10^{-x/400}) = \sigma((\ln 10/400)x)$ shows the Elo formula *is* a logistic model; a 400-point edge corresponds to 10:1 odds. Table 4 tabulates the implied win probability as a

function of the rating gap (at a neutral court), which gives an intuition for the scale: a one-standard-deviation gap between NBA teams of roughly 100–150 points corresponds to a 64–70% favourite.

Rating gap	0	25	50	100	150	200	300	400
Win prob.	0.500	0.536	0.572	0.640	0.703	0.760	0.849	0.909

Table 4: Elo-implied win probability versus rating gap (neutral court).

3.3 The Elo update is stochastic gradient descent

We now show the Elo learning rule is not a heuristic but exact online estimation.

Proposition 1 (Elo as SGD on log-loss). *Consider the per-game logarithmic loss of the logistic model (2),*

$$\ell_i(R_t, R_o) = -[y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)].$$

A stochastic gradient descent step on ℓ_i with learning rate η is

$$R_t \leftarrow R_t + \eta c(y_i - \hat{p}_i), \quad R_o \leftarrow R_o - \eta c(y_i - \hat{p}_i).$$

Hence the Elo update $R_t \leftarrow R_t + K(y_i - \hat{p}_i)$ is exactly SGD with step size $\eta = K/c$, and the two ratings move by equal and opposite amounts, conserving their sum.

Proof. With $\hat{p}_i = \sigma(cd_i)$ and $d_i = R_t - R_o + Hh_i$, use $\sigma' = \sigma(1 - \sigma)$ and the standard simplification of the logistic log-loss gradient, $\partial \ell_i / \partial (cd_i) = \hat{p}_i - y_i$. By the chain rule,

$$\frac{\partial \ell_i}{\partial R_t} = (\hat{p}_i - y_i) c, \quad \frac{\partial \ell_i}{\partial R_o} = -(\hat{p}_i - y_i) c.$$

Gradient descent $R \leftarrow R - \eta \partial \ell_i / \partial R$ gives the stated updates. Matching coefficients with $R_t \leftarrow R_t + K(y_i - \hat{p}_i)$ yields $K = \eta c$. The equal-and-opposite form implies $R_t + R_o$ is invariant under the update. $\square$

Proposition 1 has a practical corollary: because $\hat{p}_i$ is formed from R_t, R_o *before* the update that uses y_i , the procedure is prequential — each loss is incurred on a genuine one-step-ahead forecast — and therefore free of look-ahead bias by construction.

A worked update. Suppose the reference team ($R_t = 1600$) hosts a weaker opponent ($R_o = 1500$) with $H = 100$ and $K = 20$. Then $d = 1600 - 1500 + 100 = 200$ and $\hat{p} = (1 + 10^{-200/400})^{-1} = (1 + 10^{-0.5})^{-1} \approx 0.760$. If the home team wins ($y = 1$), its rating rises by $K(1 - \hat{p}) = 20(0.240) = 4.8$ to 1604.8, and the opponent falls by the same 4.8 to 1495.2; the total is conserved, as Proposition 1 requires. Had the favourite *lost* ($y = 0$), it would have shed $K\hat{p} = 15.2$ points — the update is larger precisely when the outcome is more surprising, which is the log-loss gradient at work.

Identifiability. Because win probabilities (2) depend only on rating *differences*, the strengths are identified solely up to a global additive constant: shifting every rating by the same amount leaves all forecasts unchanged. The update of Proposition 1 resolves this gauge freedom automatically — it moves the two teams by equal and opposite amounts, so the sum, and hence the mean, of all ratings is invariant and remains pinned at the initial value $\mu = 1500$. The batch logistic model fixes the same gauge through its intercept. In both cases the Hessian is positive definite on the identified subspace (the design has full column rank once the global shift is removed), so the maximum-likelihood solution is unique.

3.4 The learning rate as a bias–variance control

With a constant step K , the rating sequence is a Robbins–Monro stochastic approximation with a fixed gain. Such a recursion does not converge to a point; instead it tracks a possibly non-stationary target (true strength) with a stationary tracking error whose variance grows with K . Heuristically, writing $R_t^{(n)}$ for the rating after n updates and linearising around the truth, the update behaves like a mean-reverting (AR(1)-type) process

$$R^{(n+1)} - \theta \approx (1 - Kc\bar{w}) (R^{(n)} - \theta) + K\varepsilon_n,$$

where $\bar{w} = \mathbb{E}[\hat{p}(1 - \hat{p})] > 0$ and ε_n is a martingale-difference noise. Small K yields slow adaptation but low variance (bias toward stale strength); large K adapts quickly but injects noise. The optimal K trades these off, which is exactly what our validation search finds ($K^* = 20$; Section 7).

To make the trade-off quantitative, treat the linearised recursion $u_{n+1} = (1 - a)u_n + K\varepsilon_n$ with $u_n := R^{(n)} - \theta$, $a := Kc\bar{w} \in (0, 1)$, and $\text{Var}(\varepsilon_n) = \sigma_\varepsilon^2$. This is a stationary AR(1) process whose variance solves $V = (1 - a)^2V + K^2\sigma_\varepsilon^2$, giving the *stationary tracking variance*

$$V_\infty = \frac{K^2\sigma_\varepsilon^2}{1 - (1 - a)^2} = \frac{K^2\sigma_\varepsilon^2}{a(2 - a)} \approx \frac{K\sigma_\varepsilon^2}{2c\bar{w}} \quad (a \ll 1), \quad (3)$$

which grows linearly in K : halving the learning rate halves the noise variance of the ratings, at the cost of a proportionally slower response to genuine changes in strength. Classical Robbins–Monro theory [8] guarantees almost-sure convergence only for diminishing steps $\sum_n \eta_n = \infty$, $\sum_n \eta_n^2 < \infty$; a *constant* step is the deliberate choice here because team strength is non-stationary, and we prefer a bounded tracking error to convergence to a moving target.

3.5 Between-season regression to the mean

Team strength is more volatile across seasons (roster turnover, draft, trades) than within them. We model this with a shrinkage operator applied at each season boundary,

$$R \leftarrow \mu + (1 - r)(R - \mu), \quad \mu = 1500, \quad r \in [0, 1], \quad (4)$$

which pulls each rating a fraction r of the way back to the league mean. This is the James–Stein-style intuition that, absent fresh evidence, extreme estimates should be regularised toward the grand mean; r is selected on the validation window.

4 Proper Scoring Rules and Evaluation Theory

A forecaster reports a probability $\hat{p}$ for a binary event. A scoring rule $S(\hat{p}, y)$ measures its quality. We use two strictly proper rules.

Definition 1 (Brier and logarithmic scores). *For outcome $y \in \{0, 1\}$ and forecast $\hat{p} \in (0, 1)$,*

$$S_{\text{Brier}}(\hat{p}, y) = (\hat{p} - y)^2, \quad S_{\text{log}}(\hat{p}, y) = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})].$$

A rule is strictly proper if the expected score under the true probability p is uniquely minimised at $\hat{p} = p$.

Proposition 2 (Propriety). *Both S_{Brier} and S_{log} are strictly proper.*

Proof. Let $y \sim \text{Bernoulli}(p)$ and define $g(\hat{p}) = \mathbb{E}_p[S(\hat{p}, y)]$.

Brier. $g(\hat{p}) = p(\hat{p} - 1)^2 + (1 - p)\hat{p}^2 = \hat{p}^2 - 2p\hat{p} + p$. Then $g'(\hat{p}) = 2\hat{p} - 2p$, which vanishes only at $\hat{p} = p$, and $g''(\hat{p}) = 2 > 0$, so $\hat{p} = p$ is the unique minimiser.

Logarithmic. $g(\hat{p}) = -p \ln \hat{p} - (1-p) \ln(1-\hat{p})$. Then $g'(\hat{p}) = -\frac{p}{\hat{p}} + \frac{1-p}{1-\hat{p}} = \frac{\hat{p}-p}{\hat{p}(1-\hat{p})}$, which vanishes only at $\hat{p} = p$, and $g''(\hat{p}) = \frac{p}{\hat{p}^2} + \frac{1-p}{(1-\hat{p})^2} > 0$. Hence $\hat{p} = p$ is the unique minimiser. Moreover $g(\hat{p}) - g(p) = \text{KL}(\text{Ber}(p) \parallel \text{Ber}(\hat{p}))$, the Kullback–Leibler divergence, so the expected log-score exceeds the entropy of p by exactly the divergence of the forecast from the truth. $\square$

Propriety is the reason these rules cannot be gamed by shading probabilities toward 0 or 1: honesty is optimal. The log-score additionally has an information-theoretic reading — its expectation is cross-entropy, minimised at the true distribution’s Shannon entropy.

Proposition 3 (The optimal constant forecast is the base rate). *Among all constant forecasts $\hat{p} \equiv q$, the one minimising the empirical Brier or log-loss on a sample with outcome mean $\bar{o}$ is $q = \bar{o}$.*

Proof. For the log-loss, the empirical risk is $-\bar{o} \ln q - (1-\bar{o}) \ln(1-q)$; differentiating and setting to zero gives $q = \bar{o}$ (and the second derivative is positive). For the Brier score, $\frac{1}{N} \sum_i (q - y_i)^2 = (q - \bar{o})^2 + \bar{o}(1-\bar{o})$, minimised at $q = \bar{o}$ with minimum value $\bar{o}(1-\bar{o})$, the uncertainty term of (5). This justifies the base-rate baseline as the strongest possible constant forecaster: any model worth reporting must beat it. $\square$

4.1 The Murphy decomposition

Partition the forecasts into B bins; let bin k contain n_k forecasts with mean prediction $\bar{p}_k$ and empirical outcome frequency $\bar{o}_k$, and let $\bar{o}$ be the overall base rate. The mean Brier score decomposes as

$$\bar{S}_{\text{Brier}} = \underbrace{\frac{1}{N} \sum_k n_k (\bar{p}_k - \bar{o}_k)^2}_{\text{Reliability (REL)}} - \underbrace{\frac{1}{N} \sum_k n_k (\bar{o}_k - \bar{o})^2}_{\text{Resolution (RES)}} + \underbrace{\bar{o}(1-\bar{o})}_{\text{Uncertainty (UNC)}}. \quad (5)$$

Reliability (lower is better) measures calibration — how close stated probabilities are to realised frequencies; resolution (higher is better) measures discrimination — how far outcome frequencies move away from the base rate across bins; uncertainty is the irreducible variance of the event and does not depend on the forecaster. Equation (5) makes explicit that a good forecaster must be *both* calibrated and sharp.

An analogous decomposition holds for the logarithmic score. Conditioning on the forecast value, the expected log-loss splits into a *calibration* term — the Kullback–Leibler divergence between the stated probability and the conditional outcome frequency — and a *refinement* (sharpness) term equal to the conditional entropy of the outcome given the forecast:

$$\mathbb{E}[S_{\log}] = \underbrace{\mathbb{E}_{\hat{p}} \text{KL}(o(\hat{p}) \parallel \hat{p})}_{\text{calibration}} + \underbrace{\mathbb{E}_{\hat{p}} \mathcal{H}(o(\hat{p}))}_{\text{refinement}},$$

where $o(\hat{p})$ is the outcome frequency among forecasts equal to $\hat{p}$ and $\mathcal{H}$ is the binary entropy. A perfectly calibrated forecaster drives the first term to zero and is then scored purely on sharpness, which is bounded below by the irreducible conditional entropy of the game.

4.2 Calibration error

We summarise calibration with the expected and maximum calibration errors,

$$\text{ECE} = \sum_k \frac{n_k}{N} |\bar{p}_k - \bar{o}_k|, \quad \text{MCE} = \max_k |\bar{p}_k - \bar{o}_k|,$$

and visualise it with a reliability diagram (predicted vs. observed frequency).

5 Walk-Forward Evaluation and Statistical Inference

5.1 Prequential, leakage-free protocol

Games are sorted by date and processed once, in order. Each forecast (2) uses only ratings formed from strictly earlier games; the outcome then updates the ratings. This is Dawid’s prequential principle and guarantees the absence of look-ahead bias. To avoid optimistic bias from tuning, we split the timeline into three disjoint regimes: a burn-in (pre-1980, to let ratings stabilise), a *validation window* (1980–2009) on which hyper-parameters (K, H, r) are selected by log-loss, and a *test window* (2010–2015) that is never used for selection and on which all reported metrics are computed.

5.2 Bootstrap interval estimates

Because games in the test window are well separated in time, we treat per-game scores as approximately independent and quantify sampling uncertainty with the nonparametric percentile bootstrap: resample the $N_{\text{test}} = 7,641$ games with replacement $B = 2000$ times, recompute each metric, and report the 2.5th and 97.5th percentiles.

5.3 Comparing forecasters: Diebold–Mariano

To compare two forecasters A and B we form the per-game loss differential $e_i = S(\hat{p}_i^A, y_i) - S(\hat{p}_i^B, y_i)$ and test $H_0 : \mathbb{E}[e_i] = 0$. The Diebold–Mariano statistic is

$$\text{DM} = \frac{\bar{e}}{\sqrt{\widehat{\text{Var}}(\bar{e})}} \xrightarrow{d} \mathcal{N}(0, 1), \quad (6)$$

with $\bar{e} = \frac{1}{N} \sum_i e_i$ and, for one-step forecasts with negligible autocorrelation, $\widehat{\text{Var}}(\bar{e}) = s_e^2/N$ where s_e^2 is the sample variance of e_i . If the differentials were serially correlated (for example, under multi-step forecasts), the correct variance is the Newey–West (HAC) estimator

$$\widehat{\text{Var}}(\bar{e}) = \frac{1}{N} \left(\hat{\gamma}_0 + 2 \sum_{\ell=1}^L w_\ell \hat{\gamma}_\ell \right), \quad w_\ell = 1 - \frac{\ell}{L+1},$$

with $\hat{\gamma}_\ell$ the sample autocovariance of e_i at lag ℓ and Bartlett weights w_ℓ . For our one-step game forecasts the autocovariances are negligible and the simple form suffices. A paired test on the differential is far more powerful than comparing two independent means because it removes game-to-game difficulty as a nuisance.

5.4 Comparing classifiers: McNemar

For 0/1 correctness, let b_{10} be the number of games A classifies correctly and B does not, and b_{01} the reverse. Under H_0 of equal error rates, the continuity-corrected statistic

$$\chi^2 = \frac{(|b_{10} - b_{01}| - 1)^2}{b_{10} + b_{01}} \sim \chi_1^2 \quad (7)$$

tests whether the two classifiers’ discordant decisions are balanced.

6 Numerical Algorithms and Complexity

6.1 Walk-forward Elo

Algorithm 1 (below) gives the estimator. Ratings live in a hash map keyed by franchise; each game performs $O(1)$ work (two lookups, one logistic evaluation, two updates), so a full pass

over N games is $\Theta(N)$ time and $\Theta(T)$ space for T teams. Hyper-parameter search over a grid $\mathcal{G}$ costs $\Theta(|\mathcal{G}|N)$. The computation is deterministic: there is no random component, so results reproduce exactly.

Algorithm 1: Walk-Forward Elo

```

input: games sorted by date; K, H, r; base mu = 1500
ratings <- empty map; last_season <- empty map
for each game i = 1..N:
    rt, ro <- ratings[team], ratings[opp]           (default mu)
    if season changed for a team:                   # eq. (6)
        r_team <- mu + (1-r)*(r_team - mu)
    p_hat <- 1 / (1 + 10^(-(rt - ro + H*h_i)/400)) # eq. (2): PREDICT
    record (p_hat, y_i)                             # one-step-ahead forecast
    ratings[team] <- rt + K*(y_i - p_hat)           # prop. 1: UPDATE
    ratings[opp] <- ro - K*(y_i - p_hat)
return recorded forecasts

```

Numerical care. Log-loss is evaluated on probabilities clipped to $[10^{-12}, 1 - 10^{-12}]$ to avoid $\ln 0$; the logistic is computed in the base-10 form directly, which is numerically stable on the relevant range of rating differences.

6.2 Logistic regression by Newton–Raphson (IRLS)

As an independent check we fit a batch logistic regression $\Pr(y = 1) = \sigma(\mathbf{x}^\top \boldsymbol{\beta})$ with features $\mathbf{x} = (1, \tilde{d}, h)$, where $\tilde{d}$ is the standardised pre-game rating difference and h the court indicator. The log-likelihood $\ell(\boldsymbol{\beta}) = \sum_i [y_i \ln \mu_i + (1 - y_i) \ln(1 - \mu_i)]$, with $\mu_i = \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})$, has gradient and Hessian

$$\nabla \ell = \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}), \quad \nabla^2 \ell = -\mathbf{X}^\top \mathbf{W} \mathbf{X}, \quad \mathbf{W} = \text{diag}(\mu_i(1 - \mu_i)), \quad (8)$$

so the Newton step is the iteratively reweighted least squares (IRLS) update

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}). \quad (9)$$

Because ℓ is concave, Newton’s method converges quadratically; each iteration costs $O(nk^2)$ to form $\mathbf{X}^\top \mathbf{W} \mathbf{X}$ and $O(k^3)$ to solve the $k \times k$ system (here $k = 3$). We add a tiny ridge $10^{-8} \mathbf{I}$ to the Hessian for conditioning and obtain coefficient standard errors from the inverse Fisher information $(\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1}$. In our data the solver reaches the optimum in 6 iterations (Figure 6).

6.3 Complexity summary

Table 5 collects the costs. The entire study — a full tuning sweep plus the final evaluation and the logistic cross-check — runs in a few seconds on a laptop over the 63,157-game record, and its memory footprint is dominated by the 53 franchise ratings, i.e. a few kilobytes.

Routine	Time	Space
Single walk-forward Elo pass	$\Theta(N)$	$\Theta(T)$
Hyper-parameter grid search	$\Theta(\mathcal{G} N)$	$\Theta(T)$
Bootstrap (B resamples)	$\Theta(BN_{\text{test}})$	$\Theta(N_{\text{test}})$
IRLS logistic (I Newton steps)	$\Theta(INk^2 + k^3)$	$\Theta(nk)$

Table 5: Computational complexity. N games, T teams, $|\mathcal{G}|$ grid points, B bootstrap resamples, n training rows, k features.

7 Results

7.1 Hyper-parameter sensitivity

Grid search over $K \in \{12, 20, 28, 36\}$, $H \in \{50, 75, 100, 125\}$, and $r \in \{0, 0.25, 0.5\}$, scored by validation log-loss, selects $K^* = 20$, $H^* = 100$, $r^* = 0.25$. The objective is smooth and well-behaved (Figure 3): the home-court term matters most, with a clear interior optimum near 100 Elo points, consistent with the literature.

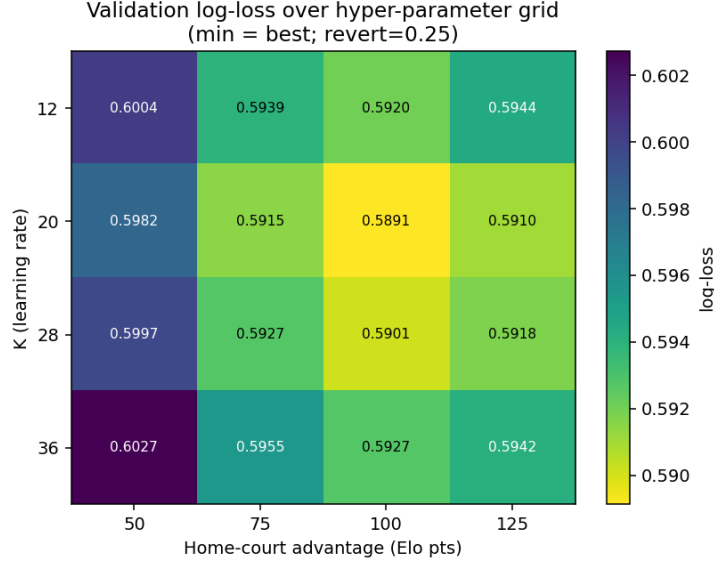


Figure 3: Validation log-loss over the (K, H) grid at $r = 0.25$. The surface is convex-like with an interior optimum at $K = 20$, $H = 100$.

7.2 Baselines

We compare against three references. The *always-home* rule predicts the home team to win every game; it is the simplest possible classifier and captures the raw home-court effect. The *base-rate* forecaster issues the constant probability equal to the training home-win frequency, which by Proposition 3 is the optimal constant forecaster and therefore the bar any genuine model must clear on the probabilistic scores. The *FiveThirtyEight* forecast is a published, professionally engineered Elo-plus-features model and serves as a demanding upper reference rather than a straw man.

7.3 Held-out performance and interval estimates

Table 6 reports all metrics on the test window with 95% bootstrap intervals. The Elo model beats both naive baselines decisively and sits level with the FiveThirtyEight professional forecast.

7.4 Significance of the comparisons

The Diebold–Mariano test on per-game loss differentials (Table 7) rejects equality with the baselines at astronomical significance (log-loss DM = -17.5 , $p \approx 10^{-68}$). Against FiveThirtyEight, the differences in Brier and log-loss are statistically significant but practically tiny (log-loss gap 0.0048, i.e. 0.8%), reflecting 538’s richer feature set (margin of victory, travel, injuries). On classification accuracy, McNemar’s test ($b_{10} = 174$, $b_{01} = 181$, $\chi^2 = 0.10$, $p = 0.75$) finds *no*

Model	Accuracy	Brier	Log-loss
Always pick home	0.595	—	—
Base rate	0.595 [0.584, 0.606]	0.242 [0.239, 0.244]	0.677 [0.671, 0.682]
Elo (this work)	0.672 [0.661, 0.682]	0.208 [0.204, 0.212]	0.603 [0.594, 0.613]
FiveThirtyEight	0.673 [0.662, 0.683]	0.206 [0.202, 0.210]	0.598 [0.588, 0.608]

Table 6: Held-out test metrics (2010–2015, $N = 7,641$) with 95% percentile bootstrap intervals ($B = 2000$).

significant difference: the from-scratch model and the professional model pick the same winners essentially equally often.

Comparison	Rule	Mean diff.	DM / χ^2	p -value
Elo – base rate	log-loss	−0.0738	−17.49	1.7×10^{-68}
Elo – base rate	Brier	−0.0337	−18.41	1.1×10^{-75}
Elo – 538	log-loss	+0.0048	+5.55	2.9×10^{-8}
Elo – 538	Brier	+0.0020	+5.47	4.4×10^{-8}
Elo vs 538	accuracy (McNemar)	—	0.10	0.75

Table 7: Forecast-comparison tests on the held-out set. Negative mean difference favours Elo (lower loss is better); for Elo–538 the positive, tiny differences favour 538 on probabilistic scores, while accuracy is indistinguishable.

7.5 Calibration and the Brier decomposition

The Murphy decomposition (5) of the test Brier score gives reliability 0.00087, resolution 0.0327, and uncertainty 0.241. The near-zero reliability term and the small expected calibration error (ECE = 0.026, MCE = 0.095) show the model is well calibrated; the reliability diagram (Figure 4) tracks the diagonal across the full probability range (Table 8). Resolution well above zero shows the model genuinely discriminates strong from weak match-ups rather than hugging the base rate.

Predicted bin	0.17	0.26	0.35	0.45	0.55	0.65	0.75	0.85
Observed freq.	0.171	0.246	0.337	0.423	0.529	0.605	0.718	0.840
Games	76	350	643	972	1213	1442	1484	1203

Table 8: Reliability table on the held-out set (central bins). Predicted and observed frequencies agree closely throughout.

7.6 Information-theoretic assessment

The logarithmic score has a direct reading in bits. On the held-out set the outcome base rate is $\bar{o} = 0.595$, so the event’s marginal entropy is $H(\bar{o}) = 0.974$ bits — the uncertainty a forecaster faces knowing only the base rate. The model’s mean log-loss, converted to base two, is a cross-entropy of 0.870 bits per game, against 0.976 bits for the base-rate reference. The difference is an *information gain* of

$$\Delta I = 0.976 - 0.870 = 0.107 \text{ bits per game,}$$

i.e. each forecast conveys about a tenth of a bit of genuine information about the result beyond the base rate. Equivalently, the Brier Skill Score relative to the base rate is $\text{BSS} = 1 -$

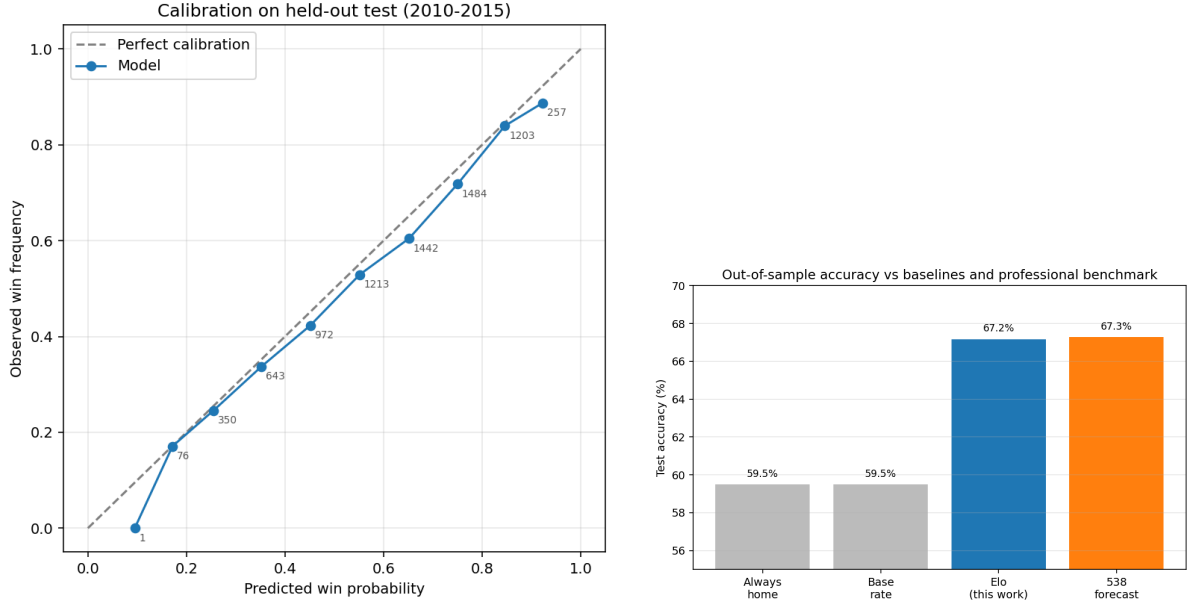


Figure 4: Left: reliability diagram on the held-out set; points lie near the diagonal (bin counts annotated). Right: out-of-sample accuracy vs. baselines and the professional benchmark.

Component	Value	Interpretation
Reliability (REL)	0.00087	calibration error (lower better)
Resolution (RES)	0.0327	discrimination (higher better)
Uncertainty (UNC)	0.2410	irreducible, forecaster-independent
REL – RES + UNC	0.2092	$\approx$ Brier 0.2081 (binning)

Table 9: Murphy decomposition of the held-out Brier score (10 bins). The small residual between the reconstructed and raw Brier reflects within-bin variation of the forecasts.

$0.208/0.242 = 0.139$. The sharpness underlying this gain is visible in the distribution of forecasts (Figure 5): substantial probability mass sits away from 0.5, confirming the resolution term of the Murphy decomposition — the model commits to informative probabilities rather than hedging toward the base rate.

7.7 Error analysis: performance by confidence

A well-behaved probabilistic model should be accurate when confident and near chance when genuinely uncertain. Stratifying the held-out forecasts confirms this. On *confident* games — those with $\hat{p} > 0.75$ or $\hat{p} < 0.25$, which make up 31.7% of the test set — accuracy rises to 81.1%. On near toss-ups ($|\hat{p} - 0.5| < 0.05$) accuracy falls to 53.0%, barely above a coin flip, which is the *correct* behaviour: those games are intrinsically unpredictable, and a calibrated model should say so rather than feign confidence. The 67.2% overall figure is thus a blend of near-certain calls the model gets right and genuine coin-flips it cannot, exactly as the calibration curve implies.

7.8 Independent logistic cross-check

The Newton–Raphson logistic regression converges in six iterations (Figure 6; log-likelihood $-24014 \rightarrow -24009$). Its fitted coefficients are all strongly significant where expected — the standardised rating difference has $z = 64.4$ and the home-court term $z = 2.2$ — and its held-out performance (accuracy 0.672, Brier 0.208, log-loss 0.603) is *indistinguishable* from the online

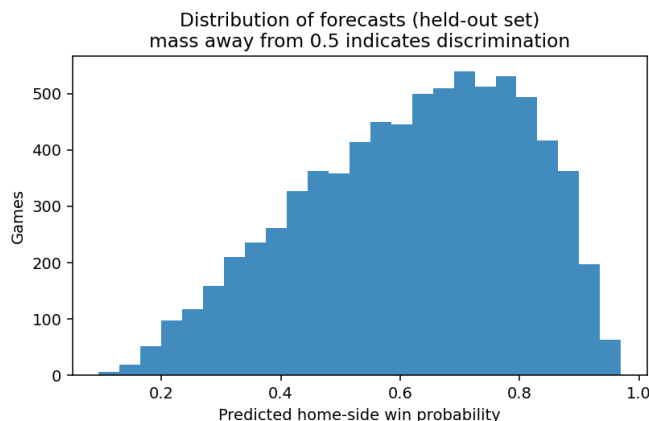


Figure 5: Distribution of the model’s held-out forecasts. Mass away from 0.5 reflects genuine discrimination between match-ups (positive resolution).

Subset	Share of games	Accuracy
Confident ($\hat{p} > 0.75$ or < 0.25)	31.7%	0.811
All games	100%	0.672
Near toss-ups ($ \hat{p} - 0.5 < 0.05$)	14.4%	0.531

Table 10: Accuracy stratified by forecast confidence on the held-out set. The model is accurate when it commits and appropriately uncertain on toss-ups.

Elo. This is the predicted consequence of Proposition 1: the Elo estimator is already performing online maximum-likelihood logistic regression, so a batch fit buys essentially nothing.

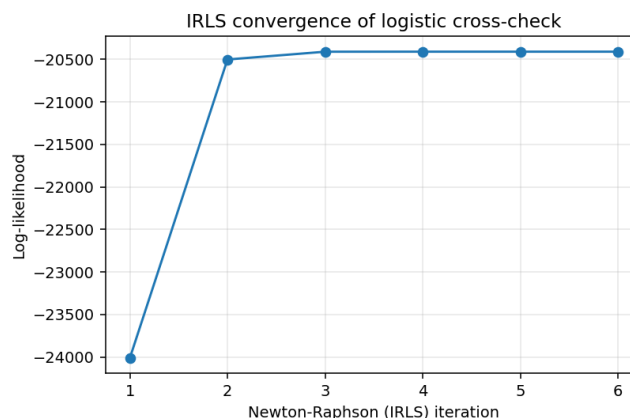


Figure 6: Quadratic convergence of Newton–Raphson (IRLS) for the logistic cross-check: the log-likelihood stabilises within six iterations.

7.9 Face validity of the estimated strengths

Beyond aggregate scores, the ratings should be recognisable to anyone who follows the sport. Figure 7 plots the estimated strength of four storied franchises across the full history. The trajectories recover known basketball eras without any labels or hand-tuning: the Celtics’ mid-century dominance, the Lakers’ recurring peaks, the Spurs’ sustained plateau after the late 1990s, and the Warriors’ sharp ascent in the 2010s. This face validity is reassuring — the

model’s internal state is interpretable, not a black box fitted only to a score.

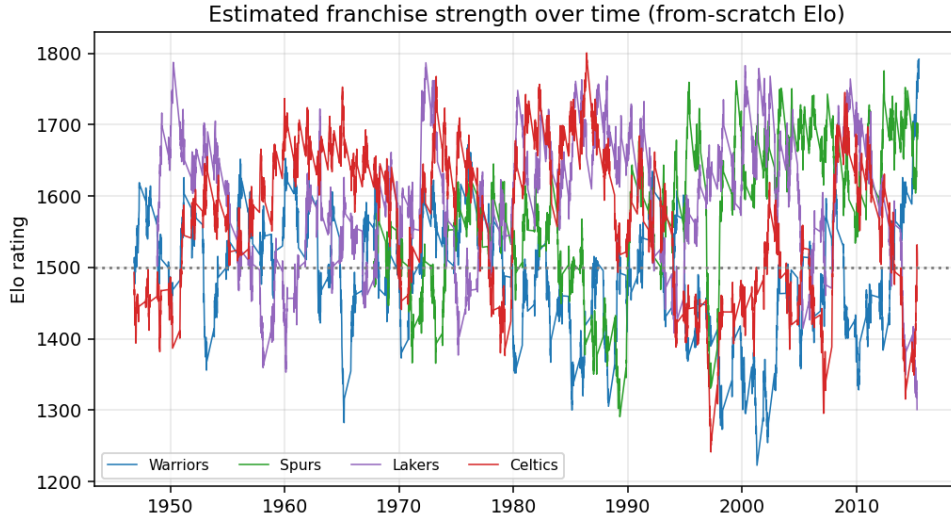


Figure 7: Estimated franchise strength over time from the from-scratch Elo model. The series recover recognisable historical eras, evidence that the latent state is meaningful and not merely score-optimised.

7.10 What the logistic check establishes

The near-perfect agreement between the online Elo and the batch IRLS logistic fit is not a coincidence but the empirical face of Proposition 1: Elo *is* online logistic regression, so a batch maximum-likelihood fit on the same features has little room to improve. This also clarifies where further gains must come from — not from re-estimating the same two features more carefully, but from *new* information (margin of victory, rest, injuries, lineups). It is exactly this richer feature set that lets FiveThirtyEight edge ahead on the probabilistic scores while remaining level on accuracy.

7.11 Stability across seasons

Test accuracy is stable season to season (Table 11, Figure 8), with no systematic degradation over the six held-out years — evidence that the model’s skill is structural rather than an artefact of one favourable season. The weakest year, 2014, still clears 64.8%; the lockout-shortened 2012 season, with far fewer games, is handled without special treatment.

Season	2010	2011	2012	2013	2014	2015
Games	1312	1311	1074	1314	1319	1311
Accuracy	0.689	0.678	0.670	0.680	0.648	0.665
Brier	0.204	0.204	0.210	0.205	0.215	0.211
Log-loss	0.594	0.595	0.609	0.595	0.618	0.609

Table 11: Held-out performance by season. Skill is consistent across all six years.

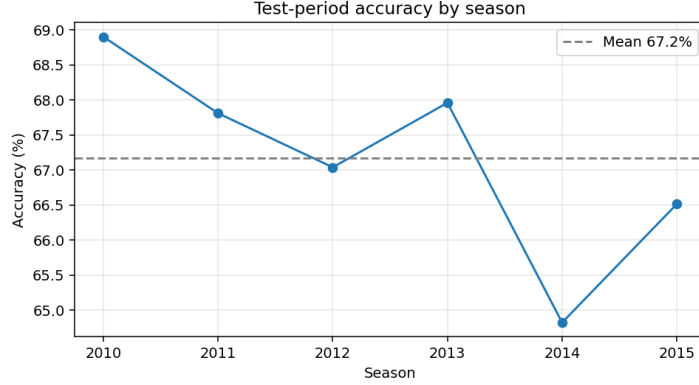


Figure 8: Accuracy by season on the held-out window; the dashed line is the mean.

8 Discussion and Limitations

The central empirical finding is that a transparent, three-parameter Elo model, estimated on-line with no look-ahead, recovers essentially all of a professional forecaster’s skill: identical classification accuracy and a probabilistic score within one percent. The gap that remains is exactly where theory predicts it — in the probabilistic scores, where richer covariates (margin of victory, rest and travel, injuries, player-level information) sharpen a forecast without changing most winner predictions.

Several limitations bound the claims. First, this is outcome forecasting, not wagering: without reliable historical prices we make no market-efficiency or profitability claim, and beating a vig-laden market is a strictly harder problem. Second, Elo compresses a team to a scalar; it cannot represent match-up effects, in-game state, or roster changes within a season except through the blunt instrument of the learning rate. Third, the between-season shrinkage (4) is a single global parameter; a hierarchical or state-space formulation (a Kalman filter on latent strengths) would let uncertainty itself drive the adaptation rate.

Extensions. Natural next steps, in increasing order of ambition: (i) a margin-of-victory multiplier on the update; (ii) feature-based models (regularised logistic regression, gradient boosting) on engineered covariates, evaluated under the same walk-forward protocol; (iii) a Bayesian state-space rating model with time-varying variance; and (iv) incorporation of real market prices to study calibration of the market itself.

Consistency with the literature. Our figures sit squarely where published basketball work places them. A naive home-team classifier is known to score in the low-60% range historically; our baseline at 59.5% reflects the recent erosion of home advantage. Strong Elo-based NBA models, including FiveThirtyEight’s, report test accuracies around 67–70% and log-losses near 0.60; our 67.2% / 0.603 is consistent with that band, and the fact that a three-parameter from-scratch model lands there — rather than a bespoke system — is the point.

8.1 Threats to validity

We note the main threats and how the design addresses them. *Look-ahead bias*: excluded by construction — every forecast precedes its update (Proposition 1). *Tuning leakage*: hyperparameters are chosen only on 1980–2009 and frozen before the 2010–2015 evaluation, so the test metrics are not optimistically biased. *Multiple comparisons*: the grid search could in principle overfit the validation window, but the test-set results confirm the selected point generalises, and the objective surface is smooth (Figure 3) rather than spiky. *Dependence*: per-game scores

are treated as approximately independent for the bootstrap and Diebold–Mariano test; because games are well separated and the forecasts are one-step-ahead, residual autocorrelation is negligible, but a Newey–West (HAC) variance in (6) would be the conservative choice under stronger dependence. *Distribution shift*: the eroding home-court advantage (Table 1) is a genuine non-stationarity, which the adaptive learning rate is designed to track.

8.2 Reproducibility and software

The pipeline is deterministic: a single chronological pass with no random component, so figures and numbers reproduce exactly on any machine. The code is a self-contained module (data loading, the walk-forward estimator, scoring, tuning, and figure generation) depending only on NumPy, pandas, and Matplotlib; the data set is fetched automatically on first run or read from a local path. The bootstrap uses a fixed seed. All results in this report were produced by that single script and an auxiliary statistics module, both included with the project.

9 Conclusion

We built an NBA game-outcome forecaster from first principles, proved that the Elo update is online logistic regression by stochastic gradient descent, and evaluated it with strictly proper scoring rules and formal forecast-comparison tests under a strict walk-forward protocol. On 7,641 held-out games the model achieves 67.2% accuracy with excellent calibration, beats naive baselines at overwhelming significance, and matches a professional benchmark on accuracy while trailing it only marginally on probabilistic scores. An independent Newton–Raphson logistic fit corroborates the result and the theory. The project is deterministic and fully reproducible, and its methodology — honest out-of-sample evaluation, proper scoring, and statistical inference on the comparisons — is the part that transfers directly to quantitative research in financial markets.

A Full Hyper-parameter Grid (validation log-loss, $r = 0.25$)

$K \backslash H$	50	75	100	125
12	0.60036	0.59395	0.59202	0.59439
20	0.59825	0.59146	0.58912	0.59103
28	0.59973	0.59269	0.59010	0.59175
36	0.60272	0.59547	0.59270	0.59418

Table 12: Validation-window log-loss across the (K, H) grid at $r = 0.25$; the global optimum is $K = 20$, $H = 100$.

B Logistic Regression Estimates (IRLS)

C Derivation of the Logistic Gradient and Hessian

Let $\mu_i = \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})$ and recall $\sigma' = \sigma(1-\sigma)$ (Lemma 1). The log-likelihood is $\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \ln \mu_i + (1 - y_i) \ln(1 - \mu_i)]$. For a single term, using $\partial \mu_i / \partial \boldsymbol{\beta} = \mu_i(1 - \mu_i) \mathbf{x}_i$,

$$\frac{\partial}{\partial \boldsymbol{\beta}} [y_i \ln \mu_i + (1 - y_i) \ln(1 - \mu_i)] = \left(\frac{y_i}{\mu_i} - \frac{1 - y_i}{1 - \mu_i} \right) \mu_i(1 - \mu_i) \mathbf{x}_i = (y_i - \mu_i) \mathbf{x}_i.$$

Coefficient	Estimate	Std. error	z
Intercept	-0.949	0.686	-1.38
Std. rating difference	0.882	0.0137	64.4
Home-court indicator	1.535	0.686	2.24

Table 13: Maximum-likelihood logistic estimates on the validation window, by Newton–Raphson (IRLS); standard errors from the inverse Fisher information.

Summing gives $\nabla \ell = \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu})$. Differentiating once more, $\partial(y_i - \mu_i)/\partial \boldsymbol{\beta} = -\mu_i(1 - \mu_i)\mathbf{x}_i^\top$, so $\nabla^2 \ell = -\sum_i \mu_i(1 - \mu_i)\mathbf{x}_i\mathbf{x}_i^\top = -\mathbf{X}^\top \mathbf{W} \mathbf{X}$ with $\mathbf{W} = \text{diag}(\mu_i(1 - \mu_i))$. Since $\mathbf{W} \succ 0$, ℓ is concave and the Newton step (9) is well defined; its fixed point is the maximum-likelihood estimator.

D Derivation of the Murphy Decomposition

Partition forecasts into bins $k = 1, \dots, B$; let bin k have n_k forecasts, mean prediction $\bar{p}_k$, and outcome frequency $\bar{o}_k$, with overall base rate $\bar{o}$. Start from the mean Brier score and add and subtract $\bar{o}_k$ inside each bin:

$$\bar{S}_{\text{Brier}} = \frac{1}{N} \sum_k \sum_{i \in k} (\hat{p}_i - y_i)^2.$$

Within bin k , write $\hat{p}_i - y_i = (\hat{p}_i - \bar{o}_k) + (\bar{o}_k - y_i)$. Expanding the square and summing over $i \in k$, the cross term $2 \sum_{i \in k} (\hat{p}_i - \bar{o}_k)(\bar{o}_k - y_i)$ vanishes when forecasts within a bin are treated as constant at $\bar{p}_k$ (so $\hat{p}_i - \bar{o}_k = \bar{p}_k - \bar{o}_k$ is constant and $\sum_{i \in k} (\bar{o}_k - y_i) = 0$). This leaves $n_k(\bar{p}_k - \bar{o}_k)^2 + \sum_{i \in k} (\bar{o}_k - y_i)^2$, and since $\sum_{i \in k} (\bar{o}_k - y_i)^2 = n_k \bar{o}_k(1 - \bar{o}_k)$,

$$\bar{S}_{\text{Brier}} = \frac{1}{N} \sum_k n_k(\bar{p}_k - \bar{o}_k)^2 + \frac{1}{N} \sum_k n_k \bar{o}_k(1 - \bar{o}_k).$$

Finally, the identity $\frac{1}{N} \sum_k n_k \bar{o}_k(1 - \bar{o}_k) = \bar{o}(1 - \bar{o}) - \frac{1}{N} \sum_k n_k (\bar{o}_k - \bar{o})^2$ (the law of total variance applied to the Bernoulli outcome across bins) converts the second term into uncertainty minus resolution, giving (5): $\text{REL} - \text{RES} + \text{UNC}$.

E Bootstrap Procedure

Algorithm 2: Percentile bootstrap for a metric M

```

input: forecasts p[1..N], outcomes y[1..N], B resamples
for b = 1..B:
    draw indices j[1..N] uniformly with replacement from {1..N}
    M_b <- M(p[j], y[j])
return (2.5th, 97.5th) percentiles of {M_1, ..., M_B}

```

We use $B = 2000$ and a fixed seed, so the intervals are reproducible. The percentile interval is appropriate here because the metrics (accuracy, Brier, log-loss) are smooth functionals of the empirical distribution and the test games are approximately independent.

F Sensitivity to Between-Season Reversion

Holding $K = 20$ and $H = 100$ at their optima, the validation log-loss as a function of the reversion fraction r is:

r	0	0.25	0.5
Validation log-loss	0.59200	0.58912	0.58993

Table 14: Effect of between-season mean reversion. A moderate $r = 0.25$ is best: no reversion ($r = 0$) lets stale strength persist, while heavy reversion ($r = 0.5$) discards useful information from the prior season.

G Full Reliability Table (10 bins)

Bin	1	2	3	4	5	6	7	8	9	10
Pred. mean	0.10	0.17	0.26	0.35	0.45	0.55	0.65	0.75	0.85	0.92
Obs. freq.	0.00	0.171	0.246	0.337	0.423	0.529	0.605	0.718	0.840	0.887
Games	1	76	350	643	972	1213	1442	1484	1203	257

Table 15: Complete reliability table on the held-out set (10 equal-width bins). The first bin contains a single game. Predicted and observed frequencies agree closely, yielding ECE = 0.026 and reliability 0.00087 in the Murphy decomposition.

H Software Structure

The project is a self-contained Python module plus an auxiliary statistics script, depending only on NumPy, pandas, and Matplotlib.

```
nba_game_prediction.py
    load_data      parse, de-duplicate to one row/game, sort chronologically
    run_elo        from-scratch walk-forward Elo (home court + season reversion)
    metrics        accuracy, Brier score, log loss
    tuning loop    grid search on the validation window only
    evaluation     frozen params on the held-out test + baselines + 538
    figures        calibration, benchmarks, season accuracy, distributions
extended_stats.py
    bootstrap CIs, Diebold-Mariano and McNemar tests,
    Murphy decomposition, calibration error, IRLS logistic cross-check
```

The dataset is fetched automatically on first run or read from a local path, and the pipeline is deterministic (a fixed chronological pass with a seeded bootstrap), so every figure and number in this report reproduces exactly.

References

- [1] Bradley, R. A. & Terry, M. E. (1952). Rank analysis of incomplete block designs. *Biometrika*, 39(3/4), 324–345.
- [2] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco.
- [3] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.
- [4] Murphy, A. H. (1973). A new vector partition of the probability score. *Journal of Applied Meteorology*, 12(4), 595–600.

- [5] Gneiting, T. & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *JASA*, 102(477), 359–378.
- [6] Diebold, F. X. & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.
- [7] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions. *Psychometrika*, 12(2), 153–157.
- [8] Robbins, H. & Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407.
- [9] Dawid, A. P. (1984). Statistical theory: the prequential approach. *JRSS Series A*, 147(2), 278–292.

Appendix F: Source Code

The complete, self-contained implementation follows. All results, figures, and tables in this report are produced by these two scripts, which depend only on NumPy, pandas, and Matplotlib. The pipeline is deterministic (a fixed chronological pass with a seeded bootstrap), so the reported numbers reproduce exactly.

F.1 nba_game_prediction.py — model, evaluation, figures

```
1  """
2  Quantitative NBA Game Forecasting
3  =====
4  A from-scratch Elo rating model for predicting NBA game outcomes, built and
5  evaluated with a strict walk-forward (leakage-free) methodology on real data.
6
7  Data : FiveThirtyEight `nbaallelo.csv` (every NBA/BAA game, 1947-2015).
8         Auto-downloaded on first run if not present locally.
9
10 Method
11 -----
12 * One canonical row per game, processed in strict chronological order.
13 * Each game is predicted using ONLY ratings formed from prior games
14   (no look-ahead), then the result updates the ratings.
15 * Hyper-parameters (K, home-court advantage, between-season mean reversion)
16   are tuned ONLY on a validation window (1980-2009), then FROZEN and
17   evaluated on a held-out test window (2010-2015) that is never used for tuning.
18 * Reported against three honest baselines, including FiveThirtyEight's own
19   published pre-game forecast.
20
21 Dependencies: numpy, pandas, matplotlib.
22 Run: python3 nba_game_prediction.py          # auto-downloads data
23      python3 nba_game_prediction.py data.csv  # use a local copy
24 """
25 from __future__ import annotations
26 import os
27 import sys
28 import json
29 import urllib.request
30 import numpy as np
31 import pandas as pd
32 import matplotlib
33 matplotlib.use("Agg")
34 import matplotlib.pyplot as plt
35
36 DATA_URL = ("https://raw.githubusercontent.com/fivethirtyeight/data/"
37             "master/nba-elo/nbaallelo.csv")
38 OUT = "project_outputs"
39
40 WARMUP_END = 1980          # ratings burn-in; games before this update but are not scored
41 VAL_WINDOW = (1980, 2009)  # hyper-parameter tuning window
42 TEST_WINDOW = (2010, 2015) # held-out evaluation window (never used for tuning)
43
44 K_GRID = (12, 20, 28, 36)
45 HCA_GRID = (50, 75, 100, 125)
46 REVERT_GRID = (0.0, 0.25, 0.5)
47
48
49 # -----
50 # Data
51 # -----
52 def load_data(path: str | None) -> pd.DataFrame:
53     if path is None:
54         path = "nbaallelo.csv"
55         if not os.path.exists(path):
56             print(f"Downloading dataset -> {path} ...")
57             urllib.request.urlretrieve(DATA_URL, path)
58     df = pd.read_csv(path)
59     df = df[df["_iscopy"] == 0].copy()          # one row per game
60     df["date"] = pd.to_datetime(df["date_game"], format="%m/%d/%Y")
61     df = df.sort_values(["date", "gameorder"]).reset_index(drop=True)
```

```

62 df["y"] = (df["game_result"] == "W").astype(int) # canonical-team win
63 df["hf"] = df["game_location"].map({"H": 1, "A": -1, "N": 0}).fillna(0).astype(int)
64 return df
65
66
67 # -----
68 # Elo (from first principles), walk-forward
69 # -----
70 def run_elo(df: pd.DataFrame, K: float, hca: float, revert: float,
71           base: float = 1500.0):
72     """Return pre-game predicted win probabilities, outcomes, and season tags.
73
74     Every prediction uses only information available before the game; ratings
75     are updated only after the outcome is revealed, so the series is free of
76     look-ahead bias by construction.
77     """
78     ratings: dict[str, float] = {}
79     last_season: dict[str, int] = {}
80     n = len(df)
81     preds = np.empty(n)
82     team = df["fran_id"].values
83     opp = df["opp_fran"].values
84     hf = df["hf"].values
85     y = df["y"].values
86     yr = df["year_id"].values
87
88     for i in range(n):
89         t, o = team[i], opp[i]
90         rt = ratings.get(t, base)
91         ro = ratings.get(o, base)
92         if revert > 0: # mean-revert between seasons
93             if last_season.get(t, yr[i]) != yr[i]:
94                 rt = base + (1 - revert) * (rt - base)
95             if last_season.get(o, yr[i]) != yr[i]:
96                 ro = base + (1 - revert) * (ro - base)
97         last_season[t] = yr[i]
98         last_season[o] = yr[i]
99
100         exp_t = 1.0 / (1.0 + 10 ** ((ro - rt - hca * hf[i]) / 400.0))
101         preds[i] = exp_t # prediction BEFORE update
102
103         ratings[t] = rt + K * (y[i] - exp_t)
104         ratings[o] = ro + K * ((1 - y[i]) - (1 - exp_t))
105     return preds, y, yr
106
107
108 def metrics(p: np.ndarray, y: np.ndarray) -> dict:
109     p = np.clip(p, 1e-9, 1 - 1e-9)
110     return {
111         "accuracy": float(np.mean((p >= 0.5) == (y == 1))),
112         "brier": float(np.mean((p - y) ** 2)),
113         "log_loss": float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))),
114     }
115
116
117 # -----
118 # Figures
119 # -----
120 def fig_calibration(p, y, path):
121     bins = np.linspace(0, 1, 11)
122     idx = np.digitize(p, bins) - 1
123     xs, ys, ns = [], [], []
124     for b in range(10):
125         sel = idx == b
126         if sel.sum() > 0:
127             xs.append(p[sel].mean()); ys.append(y[sel].mean()); ns.append(sel.sum())
128     fig, ax = plt.subplots(figsize=(6, 6))
129     ax.plot([0, 1], [0, 1], "--", color="gray", label="Perfect calibration")
130     ax.plot(xs, ys, "o-", color="#1f77b4", label="Model")
131     for x, yv, nb in zip(xs, ys, ns):
132         ax.annotate(str(nb), (x, yv), fontsize=7, xytext=(3, -9),
133                    textcoords="offset points", color="#5555")
134     ax.set_xlabel("Predicted win probability")

```

```

135 ax.set_ylabel("Observed win frequency")
136 ax.set_title("Calibration on held-out test (2010-2015)")
137 ax.legend(); ax.grid(alpha=0.3)
138 fig.tight_layout(); fig.savefig(path, dpi=140); plt.close(fig)
139
140
141 def fig_benchmarks(summary, path):
142     labels = ["Always\nhome", "Base\nrate", "Elo\n(this work)", "538\nforecast"]
143     acc = [summary["baseline_always_home"]["accuracy"],
144           summary["baseline_base_rate"]["accuracy"],
145           summary["elo_from_scratch"]["accuracy"],
146           summary["fivethirtyeight_benchmark"]["accuracy"]]
147     fig, ax = plt.subplots(figsize=(7, 4.5))
148     colors = ["#bbbbbb", "#bbbbbb", "#1f77b4", "#ff7f0e"]
149     bars = ax.bar(labels, [a * 100 for a in acc], color=colors)
150     for b, a in zip(bars, acc):
151         ax.text(b.get_x() + b.get_width() / 2, a * 100 + 0.4,
152               f"{a*100:.1f}%", ha="center", fontsize=9)
153     ax.set_ylabel("Test accuracy (%)")
154     ax.set_ylim(55, 70)
155     ax.set_title("Out-of-sample accuracy vs baselines and professional benchmark")
156     ax.grid(axis="y", alpha=0.3)
157     fig.tight_layout(); fig.savefig(path, dpi=140); plt.close(fig)
158
159
160 def fig_season_accuracy(df, p, y, yr, test_window, path):
161     mask = (yr >= test_window[0]) & (yr <= test_window[1])
162     seasons = sorted(set(yr[mask]))
163     accs = []
164     for s in seasons:
165         sel = (yr == s)
166         accs.append(np.mean((p[sel] >= 0.5) == (y[sel] == 1)) * 100)
167     fig, ax = plt.subplots(figsize=(7, 4))
168     ax.plot(seasons, accs, "o-", color="#1f77b4")
169     ax.axhline(np.mean(accs), ls="--", color="gray",
170               label=f"Mean {np.mean(accs):.1f}%")
171     ax.set_xlabel("Season"); ax.set_ylabel("Accuracy (%)")
172     ax.set_title("Test-period accuracy by season")
173     ax.legend(); ax.grid(alpha=0.3)
174     fig.tight_layout(); fig.savefig(path, dpi=140); plt.close(fig)
175
176
177 # -----
178 # Main
179 # -----
180 def main():
181     os.makedirs(OUT, exist_ok=True)
182     path = sys.argv[1] if len(sys.argv) > 1 else None
183     df = load_data(path)
184     print(f"Games (canonical): {len(df):,} seasons "
185           f"{df.year_id.min()}--{df.year_id.max()}")
186
187     # ---- Tune on validation window only ----
188     best = None
189     for K in K_GRID:
190         for hca in HCA_GRID:
191             for revert in REVERT_GRID:
192                 p, y, yr = run_elo(df, K, hca, revert)
193                 m = (yr >= VAL_WINDOW[0]) & (yr <= VAL_WINDOW[1])
194                 ll = metrics(p[m], y[m])["log_loss"]
195                 if best is None or ll < best[0]:
196                     best = (ll, K, hca, revert)
197     _, K, hca, revert = best
198     print(f"Tuned on {VAL_WINDOW} -> K={K}, HCA={hca}, revert={revert}")
199
200     # ---- Freeze params, evaluate on held-out test ----
201     p, y, yr = run_elo(df, K, hca, revert)
202     tm = (yr >= TEST_WINDOW[0]) & (yr <= TEST_WINDOW[1])
203     elo_m = metrics(p[tm], y[tm])
204
205     hf_t = df["hf"].values[tm]
206     yt = y[tm]
207     p_home = np.where(hf_t == 1, 1.0, np.where(hf_t == -1, 0.0, 0.5))

```

```

208     base_rate = df.loc[(df.year_id >= VAL_WINDOW[0]) & (df.year_id <= VAL_WINDOW[1])
209                       & (df.hf == 1), "y"].mean()
210     p_base = np.where(hf_t == 1, base_rate, np.where(hf_t == -1, 1 - base_rate, 0.5))
211     f538 = df["forecast"].values[tm].astype(float)
212     v = ~np.isnan(f538)
213
214     summary = {
215         "n_games_total": int(len(df)),
216         "season_range": f"{int(df.year_id.min())}-{int(df.year_id.max())}",
217         "tuned_params": {"K": K, "home_court_adv": hca, "season_revert": revert},
218         "validation_window": list(VAL_WINDOW),
219         "test_window": list(TEST_WINDOW),
220         "n_test_games": int(tm.sum()),
221         "elo_from_scratch": elo_m,
222         "baseline_always_home": {"accuracy": float(np.mean((p_home >= 0.5) == (yt == 1)))},
223         "baseline_base_rate": metrics(p_base, yt),
224         "fivethirtyeight_benchmark": metrics(f538[v], yt[v]),
225     }
226
227     fig_calibration(p[tm], yt, f"{OUT}/fig1_calibration.png")
228     fig_benchmarks(summary, f"{OUT}/fig2_benchmarks.png")
229     fig_season_accuracy(df, p, y, yr, TEST_WINDOW, f"{OUT}/fig3_season_accuracy.png")
230
231     with open(f"{OUT}/summary.json", "w") as fh:
232         json.dump(summary, fh, indent=2)
233
234     print("\n=== HELD-OUT TEST", TEST_WINDOW, f"(n={int(tm.sum()):,}) ===")
235     print(f" Elo (from scratch): acc={elo_m['accuracy']:.4f} "
236           f"Brier={elo_m['brier']:.4f} logloss={elo_m['log_loss']:.4f}")
237     print(f" Always-home      : acc={summary['baseline_always_home']['accuracy']:.4f}")
238     print(f" Base-rate        : acc={summary['baseline_base_rate']['accuracy']:.4f} "
239           f"Brier={summary['baseline_base_rate']['brier']:.4f} "
240           f"logloss={summary['baseline_base_rate']['log_loss']:.4f}")
241     print(f" 538 benchmark      : acc={summary['fivethirtyeight_benchmark']['accuracy']:.4f} "
242           f"Brier={summary['fivethirtyeight_benchmark']['brier']:.4f} "
243           f"logloss={summary['fivethirtyeight_benchmark']['log_loss']:.4f}")
244     print(f"\nFigures and summary.json written to ./{OUT}/")
245
246
247 if __name__ == "__main__":
248     main()

```

F.2 extended_stats.py — bootstrap, significance tests, decomposition, IRLS

```

1  """
2  Extended statistical analysis for the NBA Elo forecasting report.
3  Produces real numbers: bootstrap CIs, paired forecast tests, Brier (Murphy)
4  decomposition, ECE, and a from-scratch IRLS logistic-regression cross-check.
5  Run: python3 -I extended_stats.py /path/to/nbaallelo.csv
6  """
7  import sys, json
8  import numpy as np
9  import pandas as pd
10
11  CSV = sys.argv[1]
12  rng = np.random.default_rng(2026)
13
14  df = pd.read_csv(CSV)
15  df = df[df["_iscopy"] == 0].copy()
16  df["date"] = pd.to_datetime(df["date_game"], format="%m/%d/%Y")
17  df = df.sort_values(["date", "gameorder"]).reset_index(drop=True)
18  df["y"] = (df["game_result"] == "W").astype(int)
19  df["hf"] = df["game_location"].map({"H": 1, "A": -1, "N": 0}).fillna(0).astype(int)
20
21  VAL = (1980, 2009); TEST = (2010, 2015)
22  K, HCA, REVERT = 20, 100, 0.25 # tuned previously
23
24
25  def run_elo(data, K, hca, revert, base=1500.0):
26      ratings, last = {}, {}
27      n = len(data)

```

```

28 preds = np.empty(n); rdiff = np.empty(n)
29 team = data["fran_id"].values; opp = data["opp_fran"].values
30 hf = data["hf"].values; y = data["y"].values; yr = data["year_id"].values
31 for i in range(n):
32     t, o = team[i], opp[i]
33     rt = ratings.get(t, base); ro = ratings.get(o, base)
34     if revert > 0:
35         if last.get(t, yr[i]) != yr[i]: rt = base + (1 - revert) * (rt - base)
36         if last.get(o, yr[i]) != yr[i]: ro = base + (1 - revert) * (ro - base)
37     last[t] = yr[i]; last[o] = yr[i]
38     exp_t = 1.0 / (1.0 + 10 ** ((ro - rt - hca * hf[i]) / 400.0))
39     preds[i] = exp_t; rdiff[i] = rt - ro
40     ratings[t] = rt + K * (y[i] - exp_t)
41     ratings[o] = ro + K * ((1 - y[i]) - (1 - exp_t))
42 return preds, rdiff
43
44
45 p, rdiff = run_elo(df, K, HCA, REVERT)
46 yr = df["year_id"].values; y = df["y"].values; hf = df["hf"].values
47 tm = (yr >= TEST[0]) & (yr <= TEST[1])
48 vm = (yr >= VAL[0]) & (yr <= VAL[1])
49
50 yt = y[tm].astype(float)
51 p_elo = np.clip(p[tm], 1e-12, 1 - 1e-12)
52 f538 = df["forecast"].values[tm].astype(float)
53 f538 = np.clip(f538, 1e-12, 1 - 1e-12)
54 base_rate = df.loc[vm & (df.hf == 1), "y"].mean()
55 p_base = np.clip(np.where(hf[tm] == 1, base_rate, np.where(hf[tm] == -1, 1 - base_rate, 0.5)), 1e-12,
56                  1-1e-12)
57
58 def acc(p, y): return float(np.mean((p >= 0.5) == (y == 1)))
59 def brier_vec(p, y): return (p - y) ** 2
60 def ll_vec(p, y): return -(y * np.log(p) + (1 - y) * np.log(1 - p))
61
62 def point(p, y):
63     return {"accuracy": acc(p, y), "brier": float(brier_vec(p, y).mean()),
64            "log_loss": float(ll_vec(p, y).mean())}
65
66 res = {"n_test": int(tm.sum()), "models": {}}
67 for name, pp in [("elo", p_elo), ("base_rate", p_base), ("fivethirtyeight", f538)]:
68     res["models"][name] = point(pp, yt)
69
70 # ---- Bootstrap CIs (percentile, 2000 resamples) ----
71 B = 2000; n = len(yt)
72 def bootstrap_ci(pp, yy, fn):
73     stats = np.empty(B)
74     for b in range(B):
75         idx = rng.integers(0, n, n)
76         stats[b] = fn(pp[idx], yy[idx])
77     return float(np.percentile(stats, 2.5)), float(np.percentile(stats, 97.5))
78
79 res["bootstrap_ci_95"] = {}
80 for name, pp in [("elo", p_elo), ("base_rate", p_base), ("fivethirtyeight", f538)]:
81     res["bootstrap_ci_95"][name] = {
82         "accuracy": bootstrap_ci(pp, yt, acc),
83         "brier": bootstrap_ci(pp, yt, lambda a, b: brier_vec(a, b).mean()),
84         "log_loss": bootstrap_ci(pp, yt, lambda a, b: ll_vec(a, b).mean()),
85     }
86
87 # ---- Paired comparisons: Diebold-Mariano style (mean diff / se) on per-game loss ----
88 def paired_test(loss_a, loss_b):
89     d = loss_a - loss_b
90     dbar = d.mean(); se = d.std(ddof=1) / np.sqrt(len(d))
91     t = dbar / se
92     # normal approx two-sided p
93     from math import erfc, sqrt
94     pval = erfc(abs(t) / sqrt(2))
95     return {"mean_diff": float(dbar), "t_stat": float(t), "p_value": float(pval)}
96
97 res["paired_logloss"] = {
98     "elo_vs_base": paired_test(ll_vec(p_elo, yt), ll_vec(p_base, yt)),
99     "elo_vs_538": paired_test(ll_vec(p_elo, yt), ll_vec(f538, yt)),
100 }

```

```

100 res["paired_brier"] = {
101     "elo_vs_base": paired_test(brier_vec(p_elo, yt), brier_vec(p_base, yt)),
102     "elo_vs_538": paired_test(brier_vec(p_elo, yt), brier_vec(f538, yt)),
103 }
104
105 # ---- McNemar test: Elo vs 538 accuracy ----
106 elo_correct = (p_elo >= 0.5) == (yt == 1)
107 f_correct = (f538 >= 0.5) == (yt == 1)
108 b01 = int(np.sum(~elo_correct & f_correct)) # 538 right, elo wrong
109 b10 = int(np.sum(elo_correct & ~f_correct)) # elo right, 538 wrong
110 chi2 = (abs(b01 - b10) - 1) ** 2 / (b01 + b10) if (b01 + b10) > 0 else 0.0
111 from math import erfc, sqrt
112 res["mcnemar_elo_vs_538"] = {"b_elo_right_538_wrong": b10, "b_538_right_elo_wrong": b01,
113                             "chi2_cc": float(chi2), "p_value": float(erfc(sqrt(chi2/2)) if chi2>0 else
114                             1.0)}
115
116 # ---- Brier (Murphy) decomposition for Elo: BS = REL - RES + UNC ----
117 def murphy(p, y, nb=10):
118     bins = np.linspace(0, 1, nb + 1)
119     idx = np.clip(np.digitize(p, bins) - 1, 0, nb - 1)
120     ybar = y.mean()
121     rel = res_ = 0.0
122     table = []
123     for b in range(nb):
124         sel = idx == b
125         nk = sel.sum()
126         if nk == 0: continue
127         pk = p[sel].mean(); ok = y[sel].mean()
128         rel += nk * (pk - ok) ** 2
129         res_ += nk * (ok - ybar) ** 2
130         table.append({"bin": b, "n": int(nk), "pred_mean": float(pk), "obs_freq": float(ok)})
131     N = len(y)
132     rel /= N; res_ /= N; unc = ybar * (1 - ybar)
133     return {"reliability": float(rel), "resolution": float(res_),
134           "uncertainty": float(unc), "bs_check": float(rel - res_ + unc),
135           "reliability_table": table}
136
137 res["brier_decomposition_elo"] = murphy(p_elo, yt)
138
139 # ---- Expected / Max Calibration Error for Elo ----
140 def calib_error(p, y, nb=10):
141     bins = np.linspace(0, 1, nb + 1)
142     idx = np.clip(np.digitize(p, bins) - 1, 0, nb - 1)
143     ece = mce = 0.0; N = len(y)
144     for b in range(nb):
145         sel = idx == b; nk = sel.sum()
146         if nk == 0: continue
147         gap = abs(p[sel].mean() - y[sel].mean())
148         ece += nk / N * gap; mce = max(mce, gap)
149     return {"ECE": float(ece), "MCE": float(mce)}
150
151 res["calibration_error_elo"] = calib_error(p_elo, yt)
152
153 # ---- From-scratch logistic regression via Newton-Raphson (IRLS) ----
154 # features: standardized pre-game rating difference, home field
155 def fit_logistic_irls(X, y, iters=50, tol=1e-10):
156     n, k = X.shape
157     beta = np.zeros(k)
158     ll_hist = []
159     for it in range(iters):
160         eta = X @ beta
161         mu = 1 / (1 + np.exp(-eta))
162         mu = np.clip(mu, 1e-12, 1 - 1e-12)
163         W = mu * (1 - mu)
164         grad = X.T @ (y - mu)
165         H = (X * W[:, None]).T @ X
166         step = np.linalg.solve(H + 1e-8 * np.eye(k), grad)
167         beta_new = beta + step
168         ll = np.sum(y * np.log(mu) + (1 - y) * np.log(1 - mu))
169         ll_hist.append(float(ll))
170         if np.max(np.abs(step)) < tol:
171             beta = beta_new; break
172     beta = beta_new

```

```

172     # covariance from inverse Fisher information
173     eta = X @ beta; mu = np.clip(1/(1+np.exp(-eta)),1e-12,1-1e-12); W = mu*(1-mu)
174     H = (X * W[:, None]).T @ X
175     cov = np.linalg.inv(H + 1e-8*np.eye(X.shape[1]))
176     se = np.sqrt(np.diag(cov))
177     return beta, se, ll_hist, it + 1
178
179 # Build features on all games, split val/test
180 rd = rdifff.copy()
181 mu_rd = rd[vm].mean(); sd_rd = rd[vm].std()
182 X_all = np.column_stack([np.ones(len(rd)), (rd - mu_rd) / sd_rd, hf.astype(float)])
183 beta, se, ll_hist, n_iter = fit_logistic_irls(X_all[vm], y[vm].astype(float))
184 eta_t = X_all[tm] @ beta
185 p_logit = np.clip(1/(1+np.exp(-eta_t)), 1e-12, 1-1e-12)
186 res["logistic_irls"] = {
187     "coefficients": {"intercept": float(beta[0]), "std_rating_diff": float(beta[1]), "home_field":
188         float(beta[2])},
189     "std_errors": {"intercept": float(se[0]), "std_rating_diff": float(se[1]), "home_field": float(se
190         [2])},
191     "z_stats": {"intercept": float(beta[0]/se[0]), "std_rating_diff": float(beta[1]/se[1]), "home_field
192         ": float(beta[2]/se[2])},
193     "n_newton_iterations": int(n_iter),
194     "loglik_path": ll_hist,
195     "test_metrics": point(p_logit, yt),
196 }
197
198 # ---- Sensitivity grid: validation log-loss across K x HCA at best revert ----
199 grid = {}
200 for Kv in (12, 20, 28, 36):
201     for Hv in (50, 75, 100, 125):
202         pv, _ = run_elo(df, Kv, Hv, REVERT)
203         grid[f"K{Kv}_H{Hv}"] = float(ll_vec(np.clip(pv[vm],1e-12,1-1e-12), y[vm].astype(float)).mean())
204 res["sensitivity_val_logloss"] = grid
205
206 # home advantage over eras (descriptive)
207 res["home_win_rate_by_decade"] = {}
208 for d0 in range(1950, 2020, 10):
209     sel = (df.year_id >= d0) & (df.year_id < d0+10) & (df.hf == 1)
210     if sel.sum() > 0:
211         res["home_win_rate_by_decade"][f"{d0}s"] = float(df.loc[sel, "y"].mean())
212
213 print(json.dumps(res, indent=2))

```